

AI ASSISTED CODING

Lab 11 – Data Structures with AI: Implementing Fundamental Structures

NAME: G. Vijaya Simha Reddy

ROLLNO:2503A52671

BATCH:23

Task 1 – Stack Implementation

Loop/Data Structure Used: Stack (LIFO)

class Stack:

```
    """Stack implementation using Python list."""
```

```
    def __init__(self):
```

```
        self.items = []
```

```
    def push(self, item):
```

```
        """Add an item to the stack."""
```

```
        self.items.append(item)
```

```
    def pop(self):
```

```
        """Remove the top item."""
```

```
        if not self.is_empty():
```

```
            return self.items.pop()
```

```
        return "Stack is empty"
```

```
    def peek(self):
```

```
        """Return the top item."""
```

```
        if not self.is_empty():
```

```
        return self.items[-1]
    return "Stack is empty"
```

```
def is_empty(self):
    """Check whether stack is empty."""
    return len(self.items) == 0
```

Sample Execution

```
s = Stack()
s.push(10)
s.push(20)
s.push(30)

print("Top:", s.peak())
print("Pop:", s.pop())
print("Empty:", s.is_empty())
```

Output:

Top: 30

Pop: 30

Empty: False

Task 2 – Queue Implementation

Data Structure Used: Queue (FIFO)

```
class Queue:
    """Queue implementation using Python list."""
```

```
def __init__(self):
    self.items = []

def enqueue(self, item):
    self.items.append(item)

def dequeue(self):
    if self.items:
        return self.items.pop(0)
    return "Queue is empty"

def peek(self):
    if self.items:
        return self.items[0]
    return "Queue is empty"

def size(self):
    return len(self.items)
```

Sample Execution

```
q = Queue()
q.enqueue("A")
q.enqueue("B")
q.enqueue("C")

print(q.dequeue())
print(q.peek())
```

```
print(q.size())
```

Output:

Top: 30

Pop: 30

Empty: False

Task 3 – Singly Linked List

Python Code

```
class Node:
```

```
    """Node of a linked list."""
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.next = None
```

```
class LinkedList:
```

```
    """Singly Linked List."""
```

```
    def __init__(self):
```

```
        self.head = None
```

```
    def insert(self, data):
```

```
        new_node = Node(data)
```

```
        if self.head is None:
```

```
            self.head = new_node
```

```
    else:
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new_node

    def display(self):
        temp = self.head

        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next

        print("None")
```

Sample Execution

```
ll = LinkedList()
```

```
ll.insert(10)
```

```
ll.insert(20)
```

```
ll.insert(30)
```

```
ll.display()
```

Output:

10 -> 20 -> 30 -> None

Task 4 – Binary Search Tree (BST)

class Node:

```
def __init__(self, value):  
    self.value = value  
    self.left = None  
    self.right = None
```

class BST:

```
"""Binary Search Tree."""
```

```
def __init__(self):  
    self.root = None
```

```
def insert(self, value):  
    self.root = self._insert(self.root, value)
```

```
def _insert(self, node, value):  
    if node is None:  
        return Node(value)  
  
    if value < node.value:  
        node.left = self._insert(node.left, value)  
    else:  
        node.right = self._insert(node.right, value)  
  
    return node
```

```
def inorder(self):
    self._inorder(self.root)

def _inorder(self, node):
    if node:
        self._inorder(node.left)
        print(node.value, end=" ")
        self._inorder(node.right)
```

Sample Execution

```
tree = BST()
```

```
tree.insert(50)
```

```
tree.insert(30)
```

```
tree.insert(70)
```

```
tree.insert(20)
```

```
tree.insert(40)
```

```
tree.inorder()
```

Output:

```
20 30 40 50 70
```

Task 5 – Priority Queue

Using heapq Module

```
import heapq
```

```
class PriorityQueue:

    """Priority Queue implementation."""

    def __init__(self):
        self.heap = []

    def enqueue(self, priority, item):
        heapq.heappush(self.heap, (priority, item))

    def dequeue(self):
        if self.heap:
            return heapq.heappop(self.heap)
        return "Queue is empty"

    def display(self):
        print(self.heap)
```

Sample Execution

```
pq = PriorityQueue()

pq.enqueue(2, "Task B")
pq.enqueue(1, "Task A")
pq.enqueue(3, "Task C")

pq.display()
print(pq.dequeue())
```

Output:

```
[(1, 'Task A'), (2, 'Task B'), (3, 'Task C')]
```

```
(1, 'Task A')
```

Task 6 – Deque Implementation

Using collections.deque

```
from collections import deque
```

```
class DequeDS:
```

```
    """Double-ended queue."""
```

```
    def __init__(self):
```

```
        self.dq = deque()
```

```
    def insert_front(self, item):
```

```
        self.dq.appendleft(item)
```

```
    def insert_rear(self, item):
```

```
        self.dq.append(item)
```

```
    def remove_front(self):
```

```
        return self.dq.popleft()
```

```
    def remove_rear(self):
```

```
        return self.dq.pop()
```

Sample Execution

```
d = DequeDS()
```

```
d.insert_front(10)
```

```
d.insert_rear(20)
```

```
d.insert_front(5)
```

```
print(d.remove_front())
```

```
print(d.remove_rear())
```

Output:

5

20

Task 7 – Campus Resource Management System

The assignment asks you to choose suitable data structures for each feature and justify them.

```
from collections import deque
```

```
class CafeteriaQueue:
```

```
    """Serve students in arrival order."""
```

```
    def __init__(self):
```

```
        self.queue = deque()
```

```
    def place_order(self, student):
```

```
        self.queue.append(student)
```

```
def serve_order(self):  
    if self.queue:  
        return f"Served: {self.queue.popleft()}"  
    return "No orders."  
  
def display(self):  
    print("Waiting:", list(self.queue))
```

Sample Execution

```
cq = CafeteriaQueue()
```

```
cq.place_order("Rahul")
```

```
cq.place_order("Ananya")
```

```
cq.place_order("Kiran")
```

```
cq.display()
```

```
print(cq.serve_order())
```

```
cq.display()
```

Output:

Waiting: ['Rahul', 'Ananya', 'Kiran']

Served: Rahul

Waiting: ['Ananya', 'Kiran']

Task 8 – Smart E-Commerce Platform

The assignment asks for data-structure selection, justification, and one implementation.

```
from collections import deque
```

```
class OrderQueue:
```

```
    """Process customer orders in FIFO order."""
```

```
    def __init__(self):
```

```
        self.orders = deque()
```

```
    def place_order(self, order):
```

```
        self.orders.append(order)
```

```
    def process_order(self):
```

```
        if self.orders:
```

```
            return f"Processed: {self.orders.popleft()}"
```

```
        return "No pending orders."
```

```
    def display(self):
```

```
        print("Pending Orders:", list(self.orders))
```

```
# Sample Execution
```

```
oq = OrderQueue()
```

```
oq.place_order("Order101")
```

```
oq.place_order("Order102")
```

```
oq.place_order("Order103")
```

```
oq.display()
print(oq.process_order())
oq.display()
```

Output:

Pending Orders: ['Order101', 'Order102', 'Order103']

Processed: Order101

Pending Orders: ['Order102', 'Order103']

Conclusion

This lab demonstrates the implementation of fundamental data structures using AI-assisted Python coding. Stacks, Queues, Linked Lists, BSTs, Priority Queues, and Deques solve different real-world problems efficiently, such as attendance tracking, library management, cafeteria services, and e-commerce order processing. AI-generated code with comments and docstrings improves readability while helping students understand practical applications of each data structure.